How LLM APIs Are Served at Scale

Scaler School of Technology | ML System Design and LLMOps | Week 6 — Session 2 | Post-Lecture Review & Practice

Key Takeaways

Stateless serving is the foundation of horizontal scaling

Every OpenAI API call is stateless: the full conversation context travels in the request. This means any server can handle any request — you add capacity by adding servers. The trade-off is larger request sizes as conversations grow. Stateless design is not a simplification; it is a deliberate architectural choice with real costs.

LLM inference is GPU-memory-bound, not compute-bound

An A100 GPU can hold approximately one copy of GPT-4 (in quantised form). Serving more concurrent users means more GPUs, not faster GPUs. At 10M requests/day, you need hundreds of A100s for GPT-4 alone. Capacity planning for LLMs is fundamentally different from traditional web services.

LoRA enables multi-tenant fine-tuning without N model copies

If you have 10,000 enterprise customers, each with a fine-tuned model, you cannot load 10,000 models. LoRA solves this: one base model in GPU memory, tiny per-customer adaptor matrices (10–100MB each) loaded dynamically per request. Fine-tuning at enterprise scale is an infrastructure problem, not just an ML problem.

Rate limiting by tokens, not requests, reflects the true cost driver

An LLM API request can be 10 tokens or 10,000 tokens. Limiting by requests treats them as equivalent. Token-bucket rate limiting by token count ensures that heavy users pay proportionally for GPU resources. This is the right abstraction — and it directly drives OpenAI's pricing model.

Test Your Understanding

Try answering these without looking at your notes. Answers are on page 3.

Q1. An application sends the same 3,000-token system prompt with every API call. OpenAI's prompt caching is enabled. In a day with 100,000 API calls, estimate the token savings (input tokens only). What fraction of the application's inference cost does this represent if the typical user query is 50 tokens?

Q2. You are designing an LLM serving system for a company with 500 enterprise customers. Each customer has fine-tuned a LoRA adaptor. A request arrives — describe the complete serving path from HTTP request to response, including where the base model lives, where the LoRA adaptor is loaded from, and what happens in memory.

Q3. The OpenAI API has a rate limit of 90,000 tokens per minute (TPM). A developer's application sends requests with an average of 1,500 tokens each. What is the effective requests-per-minute (RPM) limit implied by the TPM limit? If the developer suddenly sends requests of average 3,000 tokens, how does this change?

Q4. Explain why a UK user hitting a US-East API endpoint adds approximately 100ms of latency. What physical phenomenon causes this? What architectural change eliminates it?

Design Exercises

Work through these independently or in pairs. These extend the session concepts into new problem domains.

Exercise 1: Cost Model for a RAG Application

You are building a customer support chatbot using RAG (retrieval-augmented generation). Your system prompt is 2,000 tokens. Each retrieved context chunk is 500 tokens. Each user query is 50 tokens. You expect 10,000 conversations per day, with an average of 5 turns each. Build a cost model: (a) total input tokens per day without prompt caching, (b) total input tokens per day with prompt caching (assume 80% cache hit rate on the system prompt), (c) what percentage of your input token cost is eliminated by caching. Use OpenAI's published pricing for gpt-4o-mini as your reference.

Exercise 2: Design a LoRA Serving System

You are building a platform where 1,000 small businesses can fine-tune a base LLM on their customer support data. Each business has a LoRA adaptor. Design the serving infrastructure: (a) how do you store and retrieve adaptors efficiently? (b) what is the latency cost of loading an adaptor per request? (c) how do you handle a request from a business whose adaptor is already loaded vs. one that is not? (d) draw the system diagram. Be concrete about where GPU memory, CPU memory, and disk/object storage are used.

Exercise 3: Stateless vs. Stateful Trade-off Analysis

OpenAI's API is stateless. Imagine an alternative design where the server stores conversation history (stateful). Write two paragraphs: (a) list three concrete engineering advantages of the stateful design (focus on user experience and cost), and (b) list three concrete engineering disadvantages (focus on scaling, failure recovery, and privacy). Which design would you choose for a product with 1M daily active users? Justify your answer.

Challenge Problem

The KV Cache and Context Window Economics

The KV (key-value) cache is the fundamental data structure in transformer inference. As context length grows, the KV cache grows linearly with sequence length and quadratically in attention computation. (a) If a single 128K-context GPT-4 request occupies 40GB of GPU KV cache memory, and an A100 has 80GB of total memory, how many concurrent 128K-context requests can a single A100 serve? (b) Compare this to a 4K-context request (approximately 1.25GB of KV cache). What does this imply for the pricing premium of long-context requests? (c) Propose a KV cache eviction strategy for a serving system that receives a mix of short (4K) and long (128K) context requests.

Further Reading

- **OpenAI Platform Docs — Rate Limits and Prompt Caching** — platform.openai.com/docs
  The official documentation covers token bucket rate limiting, tier structures, and prompt caching in detail. Read the rate limits and latency optimisation sections.
- **Hu et al. (2021) — LoRA: Low-Rank Adaptation of Large Language Models** — ICLR 2022, freely on arxiv.org
  The original LoRA paper. Read the abstract and Section 4 (the method). The key equation is the weight update $W = W_0 + BA$ where B and A are small matrices.
- **Andrej Karpathy — Intro to LLM (YouTube)** — youtube.com, 2023
  Karpathy's 1-hour talk covers the mechanics of LLM inference including the KV cache. Essential background for understanding why LLM serving is GPU-memory-bound.
- **vLLM: Efficient Memory Management for LLM Serving** — Kwon et al., SOSP 2023
  The paper that introduced PagedAttention for LLM serving. The engineering behind high-throughput LLM serving. Available free on arxiv.

Online Resources

- **OpenAI Cookbook** — github.com/openai/openai-cookbook — production patterns
- **vLLM Docs** — docs.vllm.ai — open-source LLM serving system
- **Cohere Docs** — docs.cohere.com — LoRA fine-tuning and deployment guides
- **LLM Inference Survey** — arxiv 2023 — comprehensive technical overview
- **Simon Willison's Blog** — simonwillison.net — practical LLM engineering
- **Lilian Weng's Blog** — lilianweng.github.io — LLM internals explained clearly

Answers

A1. Without caching: each of 100,000 calls sends 3,000 system prompt tokens + 50 user query tokens = 3,050 tokens. Total: 305M tokens. With 80% cache hit rate: 80,000 calls send only 50 tokens (system prompt cached); 20,000 calls send 3,050 tokens (cache miss). Total: 80,000 × 50 + 20,000 × 3,050 = 4M + 61M = 65M tokens. Savings: 240M tokens, which is 79% of input tokens. Since the system prompt is 3,000/3,050 = 98% of the input, caching reduces total input cost by ∼79%.

A2. The complete path: (1) HTTP request arrives at the API gateway with customer ID and prompt. (2) Gateway authenticates and routes to the LLM serving cluster. (3) Serving system looks up customer ID to identify the correct LoRA adaptor. (4) If the adaptor is not in GPU memory (cold), load it from object storage into a memory buffer (milliseconds). (5) Apply the LoRA adaptor to the base model's attention weight projections in-place. (6) Run inference: tokenise the prompt, compute forward pass with modified attention weights, generate tokens autoregressively. (7) Return the streamed response. The base model weights sit permanently in GPU VRAM; LoRA adaptors are loaded into a smaller memory region and swapped per-request or batched if multiple requests from the same customer arrive simultaneously.

A3. At 90,000 TPM with 1,500 token average: effective RPM = 90,000 / 1,500 = **60 RPM**. If request size doubles to 3,000 tokens: effective RPM = 90,000 / 3,000 = **30 RPM**. The TPM limit acts as the binding constraint when requests are large; RPM limits bind when requests are small. Developers with long-context requests are more constrained by TPM than by RPM.

A4. The speed of light through optical fibre is approximately 200,000 km/s. The UK to US distance (London to New York) is approximately 5,570 km. One-way travel time: 5,570 / 200,000 ≈ 28ms. Round-trip: ≈ 56ms. With routing overhead, TCP handshakes, and data centre processing, total added latency is ∼100ms. The architectural change is regional deployment: a EU-West serving endpoint where the model runs in a European data centre. UK users connect to EU-West (<10ms), not US-East (100ms).